Hospital Management System

Student Name: Abdul Ahad
Student ID: 24f2002963
Email: 24f2002963@ds.study.iitm.ac.in

1. Abstract

This report describes the design and implementation of a Hospital Management System, a full-stack web application that digitalises and streamlines core hospital operations. The system manages patients, doctors, appointments, treatments, and payments through a unified platform that enforces role-based access control. Three distinct user roles are provided: admins who configure and oversee the entire system, doctors who manage their schedules and patient treatment records, and patients who self-register, book appointments, and view their medical history. The backend is implemented in Python using the Flask with an SQLite relational database, Redis for caching, and Celery for asynchronous background job execution. The frontend is built with Vue.js. The system includes scheduled jobs for daily patient appointment reminders and monthly doctor activity reports delivered via email, as well as a user-triggered CSV export of treatment history. The resulting application is modular, maintainable, and demonstrates practical application of core software engineering principles including RESTful API design, role-based access control, event-driven background processing, and concurrency control.

2. Problem Statement

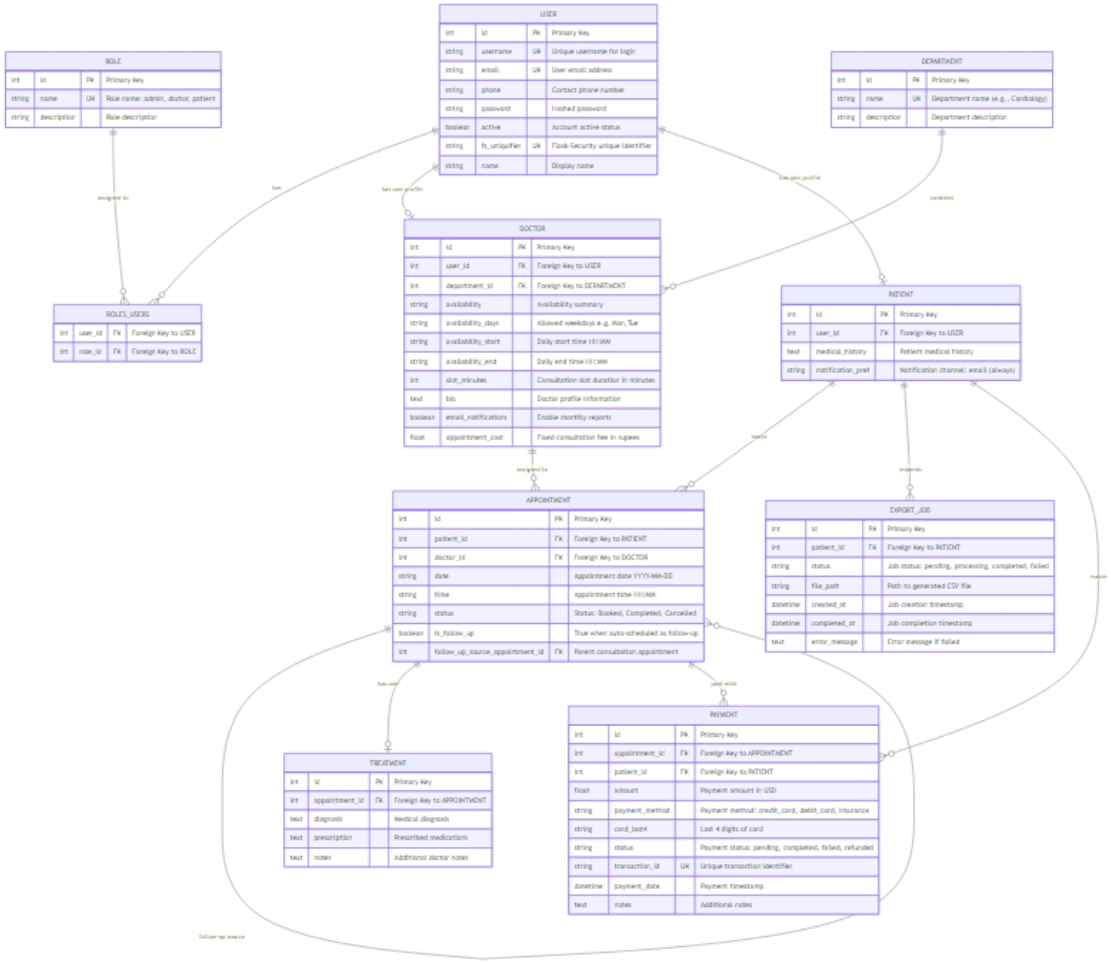
Hospitals require coordinated management of multiple entities — staff, patients, appointments, and records. This system addresses:

- **Scheduling conflicts** — centralised booking with real-time availability to prevent double-booking.
- **Disconnected records** — unified patient history across appointments.
- **Manual communication** — automated reminders, monthly reports, and export notifications via email.
- **Payment tracking** — linking consultations, amounts, and refunds.
- **Access control** — role-based permissions so each user sees only what they should.

3. Technology Stack

Component	Technology	Rationale
Backend framework	Flask (Python)	Lightweight framework
Authentication	Flask-Security	RBAC, session management
ORM	SQLAlchemy	Abstraction over SQL
Database	SQLite	Handle database
Cache and Broker	Redis	In-memory speed and task queuing
Background jobs	Celery	Distributed task queue
Email delivery	Flask-Mail	Email delivery through SMTP
PDF generation	ReportLab	Python-native PDF creation
Frontend	Vue.js 3 (CDN)	Reactive components
CSS	Bootstrap 5	Styling
Charts	Plotly	Interactive dashboard visualisations

4. Database Design



ER Diagram

4.1 Entity Overview

The database comprises nine tables: **User** (shared authentication), **Role/roles_users** (Flask-Security RBAC), **Department** (medical specialisation), **Doctor** and **Patient** (profile extensions of User), **Appointment** (booking linking Patient and Doctor), **Treatment** (diagnosis/prescription for a completed Appointment), **Payment** (financial transaction auditing including refunds), and **ExportJob** (async CSV export tracking).

4.2 Key Design Decisions

User-Profile Separation: All users share a single User table for authentication credentials. Type-specific data resides in linked Doctor and Patient profile records, simplifying credential management.

Structured Availability: Doctor availability is stored as availability_days (comma-separated weekdays), availability_start/availability_end, and slot_minutes. The backend generates valid time slots and compares them against existing bookings.

Consultation Fee per Doctor: The Doctor model includes an appointment_cost column set by the admin — the single source of truth for payment amounts.

Appointment Uniqueness Constraint: A database-level unique index on (doctor_id, date, time) prevents race conditions beyond the application-level retry logic.

Follow-up Traceability: An is_follow_up boolean and follow_up_source_appointment_id self-referencing foreign key on the Appointment table link follow-up consultations to their originals.

Payment as Audit Ledger: Positive amounts for completed transactions and negative amounts for refunds; net figures are computed with simple arithmetic. When a doctor reschedules a paid appointment, the system automatically refunds the original and transfers payment to the new appointment.

Notification Preference: The Patient model's notification_pref is always "email". All notifications are delivered via email only.

4.3 Entity Relationships

USER 1:1 DOCTOR/PATIENT (profile extension) DEPARTMENT 1:N DOCTOR PATIENT 1:N APPOINTMENT DOCTOR 1:N APPOINTMENT APPOINTMENT 1:1 TREATMENT APPOINTMENT 1:N PAYMENT (including refunds) PATIENT 1:N EXPORT_JOB APPOINTMENT self-referencing for follow-ups.

5. Implementation

5.1 Authentication and Authorisation

Flask-Security manages user sessions using cookie-based tokens. The roles_required decorator gates each route to the appropriate user type. Patient self-registration creates only patient-role users; doctor creation is exclusively an admin operation.

5.2 Appointment Booking and Serial Slot Assignment

The patient selects a doctor and date from a 7-day availability window. The backend generates all possible time slots at the configured granularity, excludes already-booked slots, and assigns the first remaining slot — ensuring fairness and preventing conflicts.

5.3 Treatment, Patient History, and Payment

When a doctor completes an appointment, they record a diagnosis, prescription, and notes as a Treatment record; a summary is appended to the patient's cumulative medical_history field. Doctors can subsequently edit their own treatment records. The payment portal simulates a gateway: the consultation fee is fixed per doctor by the admin. Cancelling a paid appointment automatically creates a negative-amount refund Payment record.

5.4 Doctor Availability and Admin Capabilities

Doctors configure their available weekdays, working hours, and slot duration via the Availability tab; saving invalidates the doctor-list cache so patients see updates immediately. Admins can also set availability when creating or editing a doctor. Admin oversight includes CRUD operations on doctors, patients, and departments; appointment filters; payment auditing; aggregate dashboard statistics; and a Doctor's Patients panel for direct patient inspection.

5.5 Patient Capabilities

Patients browse doctors as interactive profile cards (filterable by department, searchable by name) showing availability, fees, and bio. After consultation, they view their full treatment history in read-only format. Patients can export their treatment record as a CSV file. Email is mandatory at registration; all notifications are delivered via email.

6. Background Jobs and Asynchronous Processing

6.1 Architecture

Celery manages all background task execution. Redis serves as both the message broker (task queue) and result backend. A ContextTask base class injects the Flask application context into every task execution, making database sessions, configuration, and extensions available within background code.

6.2 Daily Appointment Reminders

A Celery Beat periodic task runs every day at 8:00 AM. It queries all appointments scheduled for today or tomorrow with status “Booked”. For each qualifying appointment, a reminder email is sent to the patient detailing the appointment time, doctor, and department.

6.3 Monthly Doctor Activity Report

On the first calendar day of each month, Celery Beat dispatches a task that iterates over all doctors with email notifications enabled. For each doctor, it queries all appointments in the preceding month, computes statistics (total appointments, completed, cancelled, unique patients treated), and sends an HTML email summary to the doctor’s registered address. Doctors can also download a PDF rendition of their monthly report on-demand from the Reports tab.

6.4 CSV Treatment Export

When a patient initiates an export from their dashboard, an ExportJob record is created with status “pending” and a Celery task is dispatched immediately. The task queries all completed appointments with treatment records, writes a CSV file to the exports/ directory, updates the job to “completed” with the file path, and sends an email notification to the patient.

6.5 Redis Caching

Redis provides the Flask-Caching backend. Stable reference data — admin statistics, the public doctor list, and department lists — are cached with per-endpoint TTLs of 60 seconds to 5 minutes. Frequently mutated data such as doctor appointments and patient appointment lists are deliberately served uncached to guarantee real-time accuracy after bookings, cancellations, and reschedules.

7. Security and Validation

The login_required and roles_required decorators ensure unauthenticated or unauthorised requests receive HTTP 401/403 responses, never reaching business logic. Role-based access control scopes data visibility: patients access only their own records, doctors manage only their own appointments and treatment records, and admins oversee all entities within defined operations.

8. Caching and Performance Optimisation

The application uses Redis-backed caching via Flask-Caching to reduce database query overhead for stable reference data while serving frequently mutated data directly from the database.

- **Admin Statistics** — 5-minute TTL. Aggregate counts change infrequently.
- **All Doctors List** — 60-second TTL. Includes computed upcoming availability; moderately expensive to recompute.
- **Department List** — 60-second TTL. Rarely changes after initial setup.

9. User Interface Design

The Admin Dashboard provides tabs for statistics, doctor management (including setting consultation fees), patient management, appointments, and payments. The Doctor Dashboard provides tabs for appointments (colour-coded by urgency, with upcoming future appointments highlighted in light green for quick identification), a patients list with history viewer, reports, payments, availability configuration, and a read-only profile. The Patient Dashboard provides tabs for booking (doctor profile cards with department filter and name search), appointments (with treatment detail, upcoming appointments highlighted in light green), payments, and profile management (email is the sole notification channel).

10. Development Methodology

10.1 Design Reference Process

UI design decisions were informed by examining the **NHS Digital Design System** (accessible colour and hierarchy), the **AdminLTE** Bootstrap template (tab-based admin layout), and the **Open Hospital** Java project (domain entities and relationships). All UI code was written from scratch using Bootstrap 5 and Vue.js 3; no template code was copied.

10.2 Technical Reference Sources

Official documentation and repositories consulted: Flask and Flask-Security-Too (application factory, RBAC), Celery (task queue, Beat scheduler), Vue.js 3 (Options API, reactivity), Bootstrap 5 (grid, components), SQLAlchemy (ORM, query API), and ReportLab (PDF generation).

10.3 Declaration of No AI / LLM Usage

This project — including all source code, HTML templates, CSS, JavaScript, SQL queries, test cases, and documentation — was written entirely by me without the assistance of any AI language model tools. All implementation decisions, architecture choices, algorithmic logic, and written text in this report represent my own work. External references used are cited in Section 13.

11. Demo

https://drive.google.com/file/d/1CbrkNfv0daoADyqF2xWZxWfrewv6DCtT/view?usp=sharing

12. Conclusion

The Hospital Management System successfully implements a comprehensive digital healthcare management platform. It provides role-appropriate interfaces for admins, doctors, and patients; enforces data integrity through database constraints and input validation; automates routine communications through scheduled background tasks; and demonstrates practical application of caching and asynchronous processing patterns.

13. References

1. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
2. Ronacher, A. (2010). *Flask Documentation*. Pallets Projects. https://flask.palletsprojects.com
3. Pallets Projects. (2024). *Flask source repository*. GitHub. https://github.com/pallets/flask
4. SQLAlchemy Team. (2023). *SQLAlchemy Documentation*. https://docs.sqlalchemy.org
5. SQLAlchemy Team. (2024). *SQLAlchemy source repository*. GitHub. https://github.com/sqlalchemy/sqlalchemy
6. Ask Solem and Contributors. (2023). *Celery: Distributed Task Queue Documentation*. https://docs.celeryq.dev
7. Celery Contributors. (2024). *Celery source repository*. GitHub. https://github.com/celery/celery
8. You, E. (2022). *Vue.js 3 Documentation*. https://vuejs.org/guide
9. Vue.js Core Team. (2024). *Vue.js 3 source repository*. GitHub. https://github.com/vuejs/core
10. Bootstrap Team. (2023). *Bootstrap 5 Documentation*. https://getbootstrap.com/docs/5.3
11. Bootstrap Team. (2024). *Bootstrap source repository*. GitHub. https://github.com/twbs/bootstrap
12. Redis Ltd. (2023). *Redis Documentation*. https://redis.io/documentation
13. ReportLab Group. (2023). *ReportLab User Guide*. https://www.reportlab.com/docs/reportlab-userguide.pdf
14. Flask-Security Team. (2023). *Flask-Security-Too Documentation*. https://flask-security-too.readthedocs.io
15. Flask-Security-Too Contributors. (2024). *Flask-Security-Too source repository*. GitHub. https://github.com/Flask-Security-Too/flask-security

16. PEP 8 — Style Guide for Python Code. (2001). Python Software Foundation. https://peps.python.org/pep-0008/
17. NHS England. (2024). *NHS Digital Service Manual — Design System*. https://service-manual.nhs.uk/design-system
18. Colorlib. (2024). *AdminLTE — Bootstrap Admin Dashboard Template*. GitHub. https://github.com/ColorlibHQ/AdminLTE
19. Informatici/openhospital Contributors. (2024). *Open Hospital — open-source hospital management system*. GitHub. https://github.com/informatici/openhospital
20. Bootstrap Icons Team. (2023). *Bootstrap Icons Documentation and repository*. GitHub. https://github.com/twbs/icons
21. Informatici Senza Frontiere. (2005). Open Hospital: Free and Open Source Hospital Information System. Open Hospital Project. Retrieved from https://openhospital.org
22. ColorlibHQ. (2025). AdminLTE - Free Bootstrap Admin Dashboard Template. GitHub Repository. Retrieved from https://github.com/ColorlibHQ/AdminLTE
23. NHS Digital. (2024). Design system – NHS digital service manual. NHS Digital Service Manual. Retrieved from https://service-manual.nhs.uk/design-system (service-manual.nhs.uk in Bing)